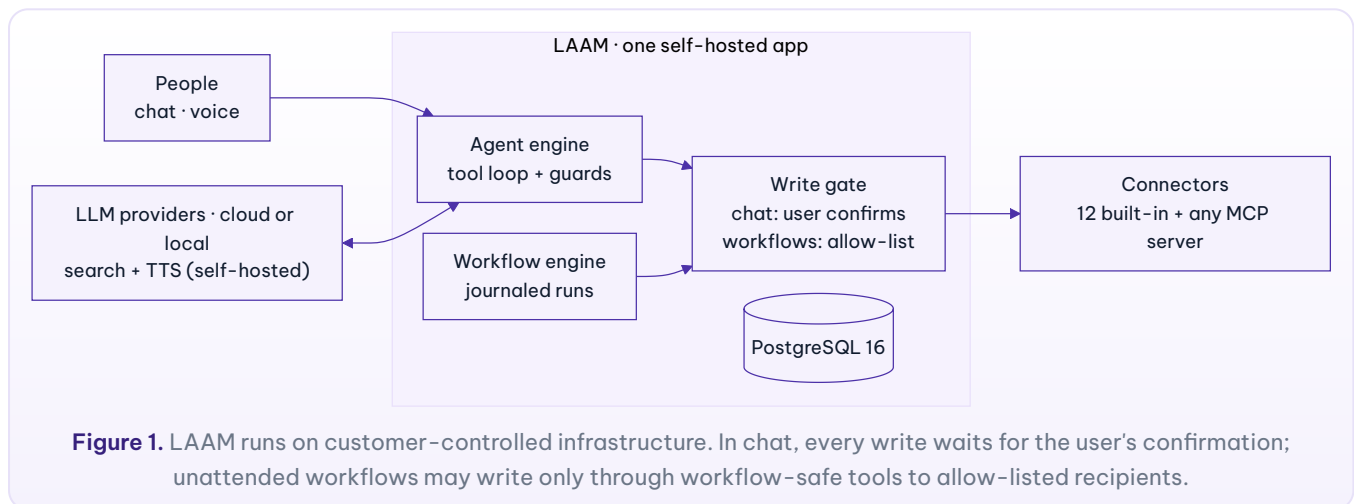


LAAM is a self-hosted, multi-user AI assistant for teams. People ask business questions in chat or by voice; LAAM answers them against **live data** by calling tools on connected systems, and in chat it **acts** on those systems only after the user confirms each write (scheduled workflows write only through workflow-safe tools, to allow-listed recipients). The same tool layer powers durable, scheduled **workflow automations** that the AI can draft, test-run and hand back for approval.

7 PRODUCT SURFACES	5 LLM PROVIDERS, ONE ROUTER	12 BUILT-IN CONNECTORS (11 INTEGRATIONS + 1 OFFLINE DEMO) · 46 TOOLS	88 API HANDLERS · 22 TABLES	3,573 AUTOMATED TEST CASES (3,567 PASSING)	1,221 COMMITTS SINCE JUN 2026
---------------------------------	---	--	--	--	---

How it works



What is technically differentiated

- **Code-enforced reliability, not prompt hope.** Each failure mode measured in testing (invented records, "success" without a write, retyped tables with errors, empty-result false negatives) has a structural guard in code.
- **Human-confirmed actions.** Any tool not declared and trusted as read-only is treated as a write. In chat, the user approves a code-built preview; the confirmed call uses the sealed arguments and its nonce is checked against the audit log.
- **Trace-grounded workflow authoring.** The AI must first observe real tool calls; a validator rejects plans that reference anything unobserved; the graph is dry-run (writes mocked, reads real) before it is offered.
- **Durable automation.** Every workflow step is journaled; runs resume after a crash; an atomic per-step claim stops a resumed run from re-executing a write.
- **Provider- and connector-agnostic.** Five LLM providers (including fully local models) behind one adapter; a generic integration layer lets any Model Context Protocol server plug in at runtime without code (a few model-facing prompt strings still reference one knowledge-graph server's tools; cleanup tracked).
- **Hands-free voice.** Streaming self-hosted neural TTS (Vietnamese and English), voice-activity barge-in, tables and charts lifted to a display panel.
- **Measured, not asserted.** An offline evaluation harness scores tool choice, arguments, grounding, restraint and write intent across repeated runs against real models.
- **Also an MCP server.** External AI agents can read LAAM's monitoring data through personal access tokens.

Where it stands today

Capability	Status	Evidence
Chat with tool calling, write confirmation, grounding guards	LIVE	Daily internal use against real connectors; run-until-done tool loop with 25-round backstop
Workflow engine (durable runs, dry-run)	LIVE	Crash-resume from journal; parallel DAG mode and cron scheduler built
AI workflow authoring (one-shot and multi-turn)	BUILT	Probe → plan → validator → compile → verify run
Voice command centre	LIVE	Self-hosted TTS; browser speech recognition (self-hosted STT planned)
MCP client · 12 built-in connectors	LIVE / BUILT	MCP servers in use; OAuth connectors activate when the operator registers apps
Access control, encrypted credentials	LIVE	4 roles; AES-256-GCM per-user credential encryption
Production deployment	PLANNED	Pre-production: self-hosted Docker Compose for development, internal use and demonstrations

Measured quality (offline evaluation, 2026-09-03)

Tool choice	Arguments	Grounding	Restraint	Termination	Write intent	Rich output
86%	97%	67%	100%	90%	100%	100%

Grader pass rates, not end-to-end accuracy: 17 synthetic scenarios × 5 runs on an open-weight model with stubbed tool responses. In a second suite, an early low-sample run suggested stable tool selection as tools grew from 4 to 60; a later 60-tool run averaged 67% — scaling is still being measured.

Technology

Application	Next.js 16, React 19, TypeScript (strict), Node 24
Data	PostgreSQL 16 (trigram search), Drizzle ORM, 19 migrations
AI	Local models via Ollama; OpenAI-compatible cloud providers; Anthropic
Integration	Model Context Protocol (client and server), OAuth connectors
Voice	Self-hosted neural TTS, in-browser voice-activity detection
Quality	Vitest (363 files), offline eval harness, no-code-execution guard test

Security and data posture

- **Self-hosted:** data stays on customer infrastructure; a fully local model configuration keeps prompts on-premises.
- **Credentials:** AES-256-GCM, per-user derived keys; secrets never reach the browser.
- **Outbound safety:** user-added MCP server URLs are checked at connect time against private and metadata ranges after DNS resolution; the web reader uses a hostname blocklist; recipient allow-lists for automated messages.
- **No code execution:** the agent can act only through declared tools, enforced by a guard test that fails in the test suite (there is no CI yet).

Next steps to production

Phase	Scope
1 · Production hardening	CI quality gates, API-wide rate limiting and CSP, access-token scope enforcement, one DNS-pinned guard for all outbound fetches, atomic nonce claim for confirmed MCP writes, pinned images, structured logging
2 · Enterprise readiness	SSO (OIDC/SAML), bring-your-own model keys with KMS, self-hosted speech-to-text, email/chat notifications
3 · Scale-out	Shared event bus and limiter store, in-product scheduler, multiple stateless app instances

Detail: 02 System Architecture · 03 Infrastructure, Security & Operations · 04 Maturity, Roadmap & IP.